```
std::shared_ptr<some_resource> resource_ptr;
void foo()
{
    if(!resource_ptr)
    {
        resource_ptr.reset(new some_resource);      <-- (1)
    }
    resource_ptr->do_something();
}
```

If the shared resource itself is safe for concurrent access, the only part that needs protecting when converting this to multithreaded code is the initialization (1), but a naïve translation such as that in the following listing can cause unnecessary serialization of threads using the resource. This is because each thread must wait on the mutex in order to check whether the resource has already been initialized.

Listing 3.11. Thread-safe lazy initialization using a mutex

```
std::shared_ptr<some_resource> resource_ptr;
std::mutex resource_mutex;
void foo()
{
    std::unique_lock<std::mutex> lk(resource_mutex);    <-- All threads are serialized here
    if(!resource_ptr)
    {
        resource_ptr.reset(new some_resource);          <-- Only the initialization needs protection
    }
    lk.unlock();
    resource_ptr->do_something();
}
```

This code is common enough, and the unnecessary serialization problematic enough, that many people have tried to come up with a better way of doing this, including the infamous *Double-Checked Locking* pattern: the pointer is first read without acquiring the lock (1) (in the code below), and the lock is acquired only if the pointer is `NULL`. The pointer is then checked *again* once the lock has been acquired (2) (hence the *double-checked* part) in case another thread has done the initialization between the first check and this thread acquiring the lock: